



INGENIERÍA MECATRÓNICA

SISTEMAS EMBEBIDOS

Actividad: U1A4. PROJECT – PHASE 1: System Architecture and HW/SW Partition

Definition of the system architecture and hardware/software partition for the project.

Alumnos:

Jassiel Obed Jiménez Ochoa

Tania Elizabeth Serrano Hernandez

Bryan Emmanuel Barboza Carrillo

Eber Jafet Rodriguez Valenciano

Docente: Osbaldo Aragón Banderas

Fecha: 2026-09-17

U1A4 - Project Phase 1: System Architecture and HW/SW Partition

SCARA Robotic Arm (JJRobots PyBot) - Embedded Systems / Robotics

Jassiel - Mechatronics Engineering, 9th semester - ITSRL

Professor: Osbaldo Aragon Banderas - Due date: September 17, 11:59 p.m.

1. Application Definition

The system is a SCARA-type robotic arm (Selective Compliance Assembly Robot Arm), built on the JJRobots PyBot platform, intended for educational pick-and-place tasks. Its main function is motion control: positioning the end effector (gripper) at specific coordinates within the work plane through the coordinated control of three joints (two rotational joints and one linear Z axis), picking up and placing small objects in a repeatable way. The system also performs a detection function (mechanical reference sensing, or homing) required to calibrate the zero position of each axis on power-up.

2. Main Components

Category	Component	Main role
MCU / SoC	ESP32-S3-DevKitC-1	Central controller: kinematics, motion control, PWM generation, Wi-Fi/USB communication
Actuators	3× NEMA17 stepper motor (shoulder, elbow, Z axis)	Positioning of the three SCARA joints
Power stage	3× A4988 stepper driver module	Current-controlled drive stage between the MCU and each stepper motor
Actuator / output	1× SG90 servo motor	End-effector (gripper) open/close control
Sensors	3× mechanical limit switch (one per axis)	Reference-position detection for the homing routine

Power supply	12 V DC / 5 A adapter + LM2596 buck converter	Main 12 V power rail for motors and a regulated 5 V rail for logic and peripherals
Communication interfaces	USB-C (UART / USB-JTAG) and integrated Wi-Fi	Programming/debug over USB; remote control and telemetry over Wi-Fi
Mechanical structure	3D-printed links, base and couplers	Physical frame of the SCARA arm (already manufactured)

3. Block Diagram

The hand-drawn sketch (Figure 1) was drawn first to define the architecture, followed by the clean digital block diagram (Figure 2), also submitted as a separate PDF file (Digital_Block_Diagram_SCARA.pdf). Both show the ESP32-S3-DevKitC-1 as the central node, the three actuation chains (A4988 driver to NEMA17 motor to limit switch) for the shoulder, elbow and Z axis, the gripper's servo motor driven by PWM, the communication interfaces (USB for programming/debug and Wi-Fi for remote control), and the two power domains (12 V for motor power and 5 V for logic/peripherals). GPIO, PWM and UART interfaces are explicitly labeled, along with the power buses.

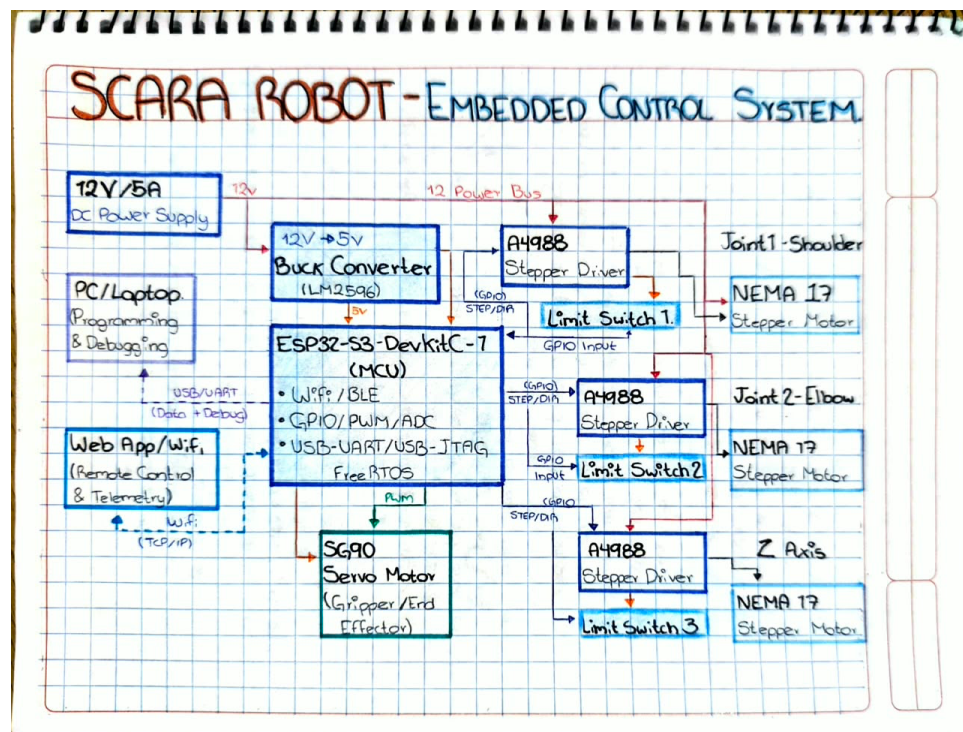


Figure 1. Hand-drawn sketch - SCARA embedded control system architecture.

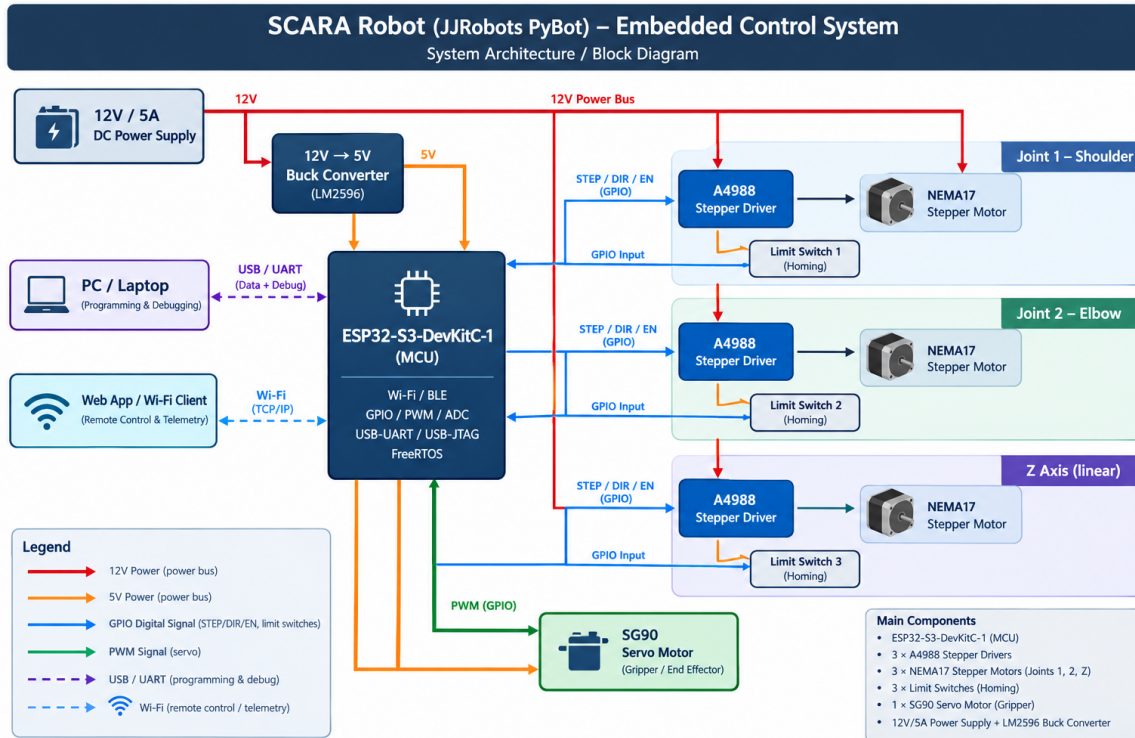


Figure 2. Digital block diagram - SCARA embedded control system architecture.

4. Hardware / Software Partition

Function	Hardware	Software / Firmware
Trajectory generation and kinematics	-	Forward/inverse kinematics of the SCARA arm, point-to-point trajectory planning and interpolation (firmware on the ESP32-S3)
Step pulse generation (STEP)	Conversion of an electrical pulse into micro-stepped motor rotation through the A4988 power stage (integrated H-bridge)	Generation of STEP/DIR pulse trains using the MCU's hardware timers (LEDC/timers) and acceleration ramps
Motor current control	Current limiting via VREF and sense resistors on the A4988; heat dissipation in the driver	Adjustment of speed/acceleration profile; ENABLE line de-asserted at rest to save power

Reference position detection (homing)	Mechanical limit switches (one per axis) with pull-up resistors	GPIO reading, software debouncing, and the homing sequence routine
Power regulation	Buck converter (LM2596) 12V to 5V; filtering capacitors at each driver	Optional voltage monitoring via ADC for low-voltage detection
End-effector control (gripper)	SG90 servo motor (gripper mechanism)	PWM signal generation (open/close angle) synchronized with the arm's motion
User communication	Physical USB-C port (UART/USB-JTAG); Wi-Fi antenna integrated in the ESP32-S3 module	Web server / command API, instruction parsing, status telemetry over Wi-Fi
Safety and operating limits	Limit switches as a last-resort physical stop	Software soft-limits per axis, watchdog timer, firmware-level emergency stop

5. Preliminary Bill of Materials (BOM)

The table below lists every field required for a complete BOM (item number, part name, description, quantity, manufacturer, part number, package/footprint, supplier, unit cost, total cost and notes), with approximate costs in USD (reference prices from online electronics retailers, subject to change by supplier and import fees). This same information is also delivered as a separate spreadsheet (BOM_Preliminary_SCARA.xlsx) as requested in the assignment format.

Item #	Part Name	Description	Qty	Manufacturer	Part Number	Package / Footprint	Supplier	Unit Cost (USD)	Total Cost (USD)	Notes
1	Microcontroller (MCU)	Dual-core Xtensa LX7 dev board, Wi-Fi/BLE, native USB-C, 240 MHz	1	Espressif Systems	ESP32-S3-DEVKITC-1-N8	Dev board, 2.54mm headers	DigiKey / AliExpress	\$15	\$15	Central controller; native USB-C simplifies programming/debug
2	Stepper motor driver	Bipolar stepper driver IC, up to 2A/phase, adjustable current via VREF	3	Allegro (IC) / Pololu (brkt.)	A4988 (StepStick module)	Breakout, 16-pin socket	AliExpress / local store	\$3	\$9	Pin-compatible with DRV8825 if a driver upgrade is needed
3	Stepper motor	1.8 deg step angle, bipolar, 1.5A/phase, ~40 N-cm holding torque	3	STEPPERONLINE (generic)	17HS4401	NEMA17 frame, 4-wire	AliExpress / local store	\$10	\$30	One motor per joint: shoulder, elbow, Z axis
4	Servo motor	9g micro servo, 180 deg rotation, 4.8-6V operating range	1	TowerPro (generic)	SG90	Micro servo housing, JST	Local store / AliExpress	\$3.50	\$3.50	Gripper actuator; upgradeable to MG90S for more torque
5	Limit switch	SPDT lever micro switch, 5A / 125VAC rated	3	Generic (Omron-compatible)	V-156-1C25	THT, lever actuator	Local electronics store	\$0.60	\$1.80	One per axis, used as the homing reference
6	Power supply	Switching AC-DC adapter, 12V output, 5A max, 5.5x2.1mm jack	1	Generic	12V5A-5525	Wall adapter	Local store / Amazon	\$12	\$12	Sized with margin above peak simultaneous motor current
7	Buck converter	Adjustable step-down DC-DC module, up to 3A output	1	Generic	LM2596 module	Breakout board, trimpot	AliExpress	\$1.50	\$1.50	Set to 5V output before connecting to the MCU
8	Electrolytic capacitor	100 uF, 35V, radial electrolytic	3	Generic	100uF/35V	THT radial, 5mm pitch	Local electronics store	\$0.25	\$0.75	Placed close to each A4988 VMOT pin for noise filtering
9	Heat sink	Small aluminum heat sink for TO-220/StepStick driver	3	Generic	HS-TO220	Adhesive-backed fin	AliExpress	\$0.60	\$1.80	Improves thermal margin under continuous operation
10	Jumper wire set	Dupont jumper wires, M-M/M-F/F-F, 40 pcs each, 20cm	1	Generic	DuPont-40x3	Ribbon cable, crimped	Local electronics store	\$5	\$5	General interconnection between MCU, drivers, switches and servo

11	Screw terminal block	2-pin screw terminal block, 5.08mm pitch	6	Generic	KF301-2P	THT, pluggable	Local electronics store	\$0.35	\$2.10	Secure power connections at each stage
12	USB-C cable	USB-A to USB-C data/power cable, 1 m	1	Generic	USB-A-C-1M	Cable assembly	Local store / Amazon	\$3.50	\$3.50	Programming/debug link between the ESP32-S3 and the PC
13	Prototype board	Perforated bakelite/phenolic prototyping board	1	Generic	PB-1015	10x15cm, single-sided	Local electronics store	\$2	\$2	For soldering the control electronics
Approximate total prototype cost									\$87.95	

6. Identified Technical Risks

#	Technical risk	Category	Potential impact	Mitigation action
1	Limited availability of key components (ESP32-S3-DevKitC-1, A4988) due to shortages or long import lead times	Component availability	Delay in prototype assembly and testing	Quote 2 alternate suppliers (local and DigiKey/AliExpress) and order with a time margin; keep a pin-compatible classic ESP32 DevKit as a fallback MCU
2	High simultaneous current draw from the 3 NEMA17 motors, exceeding the 12V/5A power supply capacity	Power consumption	Voltage drop, MCU resets, or motor step loss	Calculate peak current per axis from the A4988 VREF setting, avoid driving all 3 axes at maximum current at once in firmware, and size the supply with a $\geq 20\%$ margin
3	Overheating of the A4988 drivers during continuous operation	Temperature	Thermal shutdown reduces current output and causes loss of motion accuracy	Add heat sinks to each A4988, provide passive ventilation in the enclosure, and limit current to the minimum needed for the required torque
4	Electrical noise (EMI) from the PWM switching of the drivers affecting limit-switch readings or communication	Electrical noise / EMC	False homing triggers or intermittent communication errors	Physically separate power and signal wiring/traces, use shielded cable for the limit switches, and add decoupling capacitors near each driver
5	Incompatibility of stepper-pulse libraries with the ESP32-S3 dual-core and its specific timer peripherals	Firmware	Irregular motion or jitter in the motors	Validate the step-generation library (e.g., FastAccelStepper) on the ESP32-S3 with a bench test before integrating it into the full project

6	Loss of Wi-Fi connection during remote control of the arm	Communication	Commands not received or arm stopped mid-trajectory	Implement a safety timeout that stops the motors if no heartbeat is received from the client within X seconds, and allow a local USB control fallback
---	---	---------------	---	---

Analysis Questions

1. Which component is the most important part of your system and why?

The ESP32-S3-DevKitC-1 microcontroller is the most critical component, since it is the only element in the system with no immediate functional substitute: it concentrates the kinematics calculations, the control signal generation for all three joints and the servo, the reading of the homing sensors, and all communication with the user. A failure in the MCU stops the entire system, whereas the failure of a single driver or motor only affects one axis.

2. Which function should be performed by software instead of hardware?

Inverse kinematics (converting a desired end-effector position into the angles/displacement of each joint) and trajectory planning with acceleration ramps should be solved in software. Implementing them in dedicated hardware (for example, fixed logic or a specific coprocessor) would add cost and design complexity with no real benefit, since the ESP32-S3 has more than enough computing capacity to solve them in real time.

3. What is the main technical risk of your project?

The risk with the highest combined probability and impact is the current draw of the three NEMA17 motors operating simultaneously, since it can cause voltage drops that reset the MCU or cause step loss, affecting the accuracy of the entire arm. It is a cross-cutting risk that touches the electrical design, the power supply sizing, and the firmware logic.

4. What would happen if one of the selected components becomes unavailable?

If the ESP32-S3-DevKitC-1 became unavailable, the project could migrate to a classic ESP32 DevKit (WROOM-32) without redesigning the whole architecture, since it shares GPIO, PWM and Wi-Fi connectivity, although native USB and some performance would be lost. If the A4988 driver became unavailable, the design already allows a direct replacement with a DRV8825 (same pinout, higher current rating), which has also already been used in the team's earlier minisumo robot.

5. What part of the architecture could be simplified to reduce cost or power consumption?

The Z axis could be simplified by removing its dedicated stepper motor and replacing it with a manually actuated mechanism or a lower-cost servo motor if the application does not require continuous vertical positioning, cutting the number of drivers and idle current draw by a third. Another option is to replace the per-axis limit switch with a single shared sensor used in a sequential homing routine, reducing wiring and GPIO usage at the cost of a slightly slower homing sequence.